

Bayescoverage APP - overview

Leontine Alkema, May 11 2026

Installation info is given in the readme.

Just loading all packages here to avoid issues (we may be able to remove some, eg brms).

```
library(bayescoveragemodel)
library(localhierarchy) # for check_nas...
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
#library(ggnewscale)
#library(ggpubr)
library(haven)
#library(stringr)
#library(readxl)
library(brms) # brms required for routine stuff
```

Loading required package: Rcpp

Loading 'brms' package (version 2.23.0). Useful instructions
can be found by typing `help('brms')`. A more detailed introduction
to the package is available through `vignette('brms_overview')`.

Attaching package: 'brms'

The following object is masked from 'package:stats':

ar

```
options(cmdstanr_warn_inits = FALSE)
```

Where data are stored:

```
data_folder <- "data_raw" # for survey data
routinedata_dir <- here::here("private/routinedata/")
# see note on working with routine dummy data below
```

Choose an indicator. Naming comes from survey data, options include

- c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4") , with routine as available
- c("bfexcl0_5", "ancq8", "cci", "sba"), no routine data

```
indicator_select <- "anc4" #"cci"#"vdpt"#"anc4"
```

Read in survey data and process

```
dat0 <- read_dta(here::here(data_folder, "ICEH_all.long.dta")) %>%
  rename(r = r_raw, se = se_raw)
# process for indicator_select
dat <- process_data(
  dat = dat0,
  regions_dat = bayescoveragemodel::regions_all,
  indicator = indicator_select)
```

```
[1] "For ANC4, we combined anc42 and anc43 and call it anc4."
[1] "Some clusters are NA, these observations are excluded for now."
[1] "Observations with missing clusters"
# A tibble: 6 x 3
  iso    year cluster
<chr> <dbl> <chr>
1 XKX    2013 <NA>
2 XKX    2019 <NA>
```

```

3 PBA      2010 <NA>
4 PPU      2011 <NA>
5 PIL      2006 <NA>
6 PIL      2011 <NA>
[1] "Minor T0 D0: Figure out exact reference years, ie different reference period for anc4."
[1] "Some SEs are NA, these observations are excluded for now."
[1] "T0 D0: add imputation for missing SEs."
[1] "We add record_id_fixed to the data."
[1] "Currently no biases assigned."
[1] "Column possible_outlier not included in data, no pre-defined outliers used"
[1] "Remove data that's not DHS or MICS, number of obs removed:"
[1] 6

```

Fitting for one country, use ISO to select

```

# countries with survey data
# unique(dat$iso)
iso_select <- "RWA"
dat_use <-
  dat %>%
  filter(iso %in% iso_select)

```

Do we want to add routine data?

```

add_routine <- ifelse(
  indicator_select %in% c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4"),
  TRUE,
  FALSE)
add_routine

```

```
[1] TRUE
```

If yes, read in routine data.

```

if (!add_routine){
  routine_dat <- routine_dat_use <- NULL
} else {
  # if we want to use real data
  # Some processing, we'll move this processing into a function.
  # routine_dat0 <- read_csv(
  #   paste0(routinedata_dir, "/dat_combi_",

```

```

#       indicator_select, ".csv")) %>%
#   mutate(indicator = indicator_select)
# # routine_dat has indicator_name (name in routine) and
# # indicator (indicator name from survey data)
# routine_dat <-
#   routine_dat0 %>%
#   filter(worst_notmissing >= 90,
#          worst_notoutliers >= 90,
#          worst_abs_diff_level < 0.2,
#          worst_rr >= 60) %>%
#   #>%
#   # filter(!is.na(diff_roc))
#   # remove countries w/o any nonNA diff_roc
#   group_by(iso) %>%
#   filter(sum(!is.na(diff_roc)) > 0) %>%
#   ungroup() %>%
#   mutate(sd_routine_roc = 0,
#          sd_routine = 0) %>%
#   mutate(worst_combi =
#          worst_rr/100*worst_notmissing/100*worst_notoutliers)

# routine_dat_use <-
#   routine_dat %>%
#   filter(iso %in% iso_select)

# or if we want to use dummy data
routine_dat_use <- read_csv(here::here("data_raw/routine_toydata.csv")) %>%
# add iso
mutate(iso = iso_select,
# add indicator name (as per routine data naming)
       indicator_name = case_when(
         indicator_select == "anc1trimester" ~ "anc",
         indicator_select == "vmsl" ~ "measles2",
         indicator_select == "ideliv" ~ "instdeliveries",
         indicator_select == "vdpt" ~ "penta3",
         indicator_select == "anc4" ~ "anc4"
       ))
}

```

Rows: 5 Columns: 9

-- Column specification -----

Delimiter: ","

dbl (6): year, routine_value, routine_roc, worst_combi, sd_routine_roc, sd_r...

lgl (3): iso, indicator_name, indicator

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
fit <- fit_model(runstep = "local_national",
  y = "invprobit_indicator",
  se = "se_invprobit_indicator",
  survey_df = dat_use,
  routine_data = routine_dat_use,
  #stan_file_path = stan_file_path_update,
  chains = 4,
  iter_sampling = 300,
  iter_warmup = 150,
  # validation_cutoff_year = 2010,
  get_posteriors = TRUE)
```

```
[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"
[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We filter data to be inside estimation period"
[1] "We use national data"
[1] "NOT using or producing aggregates"
```

Joining with `by = join\_by(iso)`

Running MCMC with 4 parallel chains...

```
Chain 1 Iteration: 1 / 450 [ 0%] (Warmup)
Chain 1 Iteration: 10 / 450 [ 2%] (Warmup)
Chain 1 Iteration: 20 / 450 [ 4%] (Warmup)
```

Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
Chain 1 Exception: normal_lpdf: Location parameter is nan, but must be finite! (in '/var/folders/c63b51eb145d.stan', line 401, column 10 to column 47)
Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like real, it is probably not a problem.
Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
Chain 1

Chain 2 Iteration: 1 / 450 [0%] (Warmup)
Chain 2 Iteration: 10 / 450 [2%] (Warmup)
Chain 2 Iteration: 20 / 450 [4%] (Warmup)
Chain 3 Iteration: 1 / 450 [0%] (Warmup)
Chain 3 Iteration: 10 / 450 [2%] (Warmup)
Chain 3 Iteration: 20 / 450 [4%] (Warmup)

Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
Chain 3 Exception: normal_lpdf: Location parameter is nan, but must be finite! (in '/var/folders/c63b51eb145d.stan', line 401, column 10 to column 47)
Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like real, it is probably not a problem.
Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
Chain 3

Chain 4 Iteration: 1 / 450 [0%] (Warmup)
Chain 4 Iteration: 10 / 450 [2%] (Warmup)
Chain 4 Iteration: 20 / 450 [4%] (Warmup)
Chain 4 Iteration: 30 / 450 [6%] (Warmup)
Chain 1 Iteration: 30 / 450 [6%] (Warmup)
Chain 1 Iteration: 40 / 450 [8%] (Warmup)
Chain 1 Iteration: 50 / 450 [11%] (Warmup)
Chain 1 Iteration: 60 / 450 [13%] (Warmup)
Chain 2 Iteration: 30 / 450 [6%] (Warmup)
Chain 2 Iteration: 40 / 450 [8%] (Warmup)
Chain 2 Iteration: 50 / 450 [11%] (Warmup)
Chain 3 Iteration: 30 / 450 [6%] (Warmup)
Chain 3 Iteration: 40 / 450 [8%] (Warmup)
Chain 4 Iteration: 40 / 450 [8%] (Warmup)
Chain 4 Iteration: 50 / 450 [11%] (Warmup)
Chain 1 Iteration: 70 / 450 [15%] (Warmup)
Chain 1 Iteration: 80 / 450 [17%] (Warmup)
Chain 1 Iteration: 90 / 450 [20%] (Warmup)
Chain 2 Iteration: 60 / 450 [13%] (Warmup)

Chain 2 Iteration: 70 / 450 [15%] (Warmup)
Chain 3 Iteration: 50 / 450 [11%] (Warmup)
Chain 3 Iteration: 60 / 450 [13%] (Warmup)
Chain 3 Iteration: 70 / 450 [15%] (Warmup)
Chain 4 Iteration: 60 / 450 [13%] (Warmup)
Chain 4 Iteration: 70 / 450 [15%] (Warmup)
Chain 4 Iteration: 80 / 450 [17%] (Warmup)
Chain 1 Iteration: 100 / 450 [22%] (Warmup)
Chain 1 Iteration: 110 / 450 [24%] (Warmup)
Chain 1 Iteration: 120 / 450 [26%] (Warmup)
Chain 1 Iteration: 130 / 450 [28%] (Warmup)
Chain 1 Iteration: 140 / 450 [31%] (Warmup)
Chain 1 Iteration: 150 / 450 [33%] (Warmup)
Chain 1 Iteration: 151 / 450 [33%] (Sampling)
Chain 1 Iteration: 160 / 450 [35%] (Sampling)
Chain 2 Iteration: 80 / 450 [17%] (Warmup)
Chain 2 Iteration: 90 / 450 [20%] (Warmup)
Chain 2 Iteration: 100 / 450 [22%] (Warmup)
Chain 3 Iteration: 80 / 450 [17%] (Warmup)
Chain 3 Iteration: 90 / 450 [20%] (Warmup)
Chain 3 Iteration: 100 / 450 [22%] (Warmup)
Chain 4 Iteration: 90 / 450 [20%] (Warmup)
Chain 4 Iteration: 100 / 450 [22%] (Warmup)
Chain 4 Iteration: 110 / 450 [24%] (Warmup)
Chain 4 Iteration: 120 / 450 [26%] (Warmup)
Chain 4 Iteration: 130 / 450 [28%] (Warmup)
Chain 4 Iteration: 140 / 450 [31%] (Warmup)
Chain 4 Iteration: 150 / 450 [33%] (Warmup)
Chain 4 Iteration: 151 / 450 [33%] (Sampling)
Chain 1 Iteration: 170 / 450 [37%] (Sampling)
Chain 1 Iteration: 180 / 450 [40%] (Sampling)
Chain 1 Iteration: 190 / 450 [42%] (Sampling)
Chain 1 Iteration: 200 / 450 [44%] (Sampling)
Chain 1 Iteration: 210 / 450 [46%] (Sampling)
Chain 1 Iteration: 220 / 450 [48%] (Sampling)
Chain 1 Iteration: 230 / 450 [51%] (Sampling)
Chain 2 Iteration: 110 / 450 [24%] (Warmup)
Chain 2 Iteration: 120 / 450 [26%] (Warmup)
Chain 2 Iteration: 130 / 450 [28%] (Warmup)
Chain 2 Iteration: 140 / 450 [31%] (Warmup)
Chain 2 Iteration: 150 / 450 [33%] (Warmup)
Chain 2 Iteration: 151 / 450 [33%] (Sampling)
Chain 3 Iteration: 110 / 450 [24%] (Warmup)

Chain 3 Iteration: 120 / 450 [26%] (Warmup)
Chain 3 Iteration: 130 / 450 [28%] (Warmup)
Chain 3 Iteration: 140 / 450 [31%] (Warmup)
Chain 3 Iteration: 150 / 450 [33%] (Warmup)
Chain 3 Iteration: 151 / 450 [33%] (Sampling)
Chain 3 Iteration: 160 / 450 [35%] (Sampling)
Chain 3 Iteration: 170 / 450 [37%] (Sampling)
Chain 4 Iteration: 160 / 450 [35%] (Sampling)
Chain 4 Iteration: 170 / 450 [37%] (Sampling)
Chain 4 Iteration: 180 / 450 [40%] (Sampling)
Chain 4 Iteration: 190 / 450 [42%] (Sampling)
Chain 4 Iteration: 200 / 450 [44%] (Sampling)
Chain 4 Iteration: 210 / 450 [46%] (Sampling)
Chain 4 Iteration: 220 / 450 [48%] (Sampling)
Chain 1 Iteration: 240 / 450 [53%] (Sampling)
Chain 1 Iteration: 250 / 450 [55%] (Sampling)
Chain 1 Iteration: 260 / 450 [57%] (Sampling)
Chain 1 Iteration: 270 / 450 [60%] (Sampling)
Chain 1 Iteration: 280 / 450 [62%] (Sampling)
Chain 1 Iteration: 290 / 450 [64%] (Sampling)
Chain 1 Iteration: 300 / 450 [66%] (Sampling)
Chain 1 Iteration: 310 / 450 [68%] (Sampling)
Chain 2 Iteration: 160 / 450 [35%] (Sampling)
Chain 2 Iteration: 170 / 450 [37%] (Sampling)
Chain 2 Iteration: 180 / 450 [40%] (Sampling)
Chain 2 Iteration: 190 / 450 [42%] (Sampling)
Chain 3 Iteration: 180 / 450 [40%] (Sampling)
Chain 3 Iteration: 190 / 450 [42%] (Sampling)
Chain 3 Iteration: 200 / 450 [44%] (Sampling)
Chain 3 Iteration: 210 / 450 [46%] (Sampling)
Chain 3 Iteration: 220 / 450 [48%] (Sampling)
Chain 3 Iteration: 230 / 450 [51%] (Sampling)
Chain 3 Iteration: 240 / 450 [53%] (Sampling)
Chain 4 Iteration: 230 / 450 [51%] (Sampling)
Chain 4 Iteration: 240 / 450 [53%] (Sampling)
Chain 4 Iteration: 250 / 450 [55%] (Sampling)
Chain 4 Iteration: 260 / 450 [57%] (Sampling)
Chain 4 Iteration: 270 / 450 [60%] (Sampling)
Chain 4 Iteration: 280 / 450 [62%] (Sampling)
Chain 4 Iteration: 290 / 450 [64%] (Sampling)
Chain 4 Iteration: 300 / 450 [66%] (Sampling)
Chain 1 Iteration: 320 / 450 [71%] (Sampling)
Chain 1 Iteration: 330 / 450 [73%] (Sampling)

Chain 1 Iteration: 340 / 450 [75%] (Sampling)
Chain 1 Iteration: 350 / 450 [77%] (Sampling)
Chain 1 Iteration: 360 / 450 [80%] (Sampling)
Chain 1 Iteration: 370 / 450 [82%] (Sampling)
Chain 1 Iteration: 380 / 450 [84%] (Sampling)
Chain 2 Iteration: 200 / 450 [44%] (Sampling)
Chain 2 Iteration: 210 / 450 [46%] (Sampling)
Chain 2 Iteration: 220 / 450 [48%] (Sampling)
Chain 2 Iteration: 230 / 450 [51%] (Sampling)
Chain 2 Iteration: 240 / 450 [53%] (Sampling)
Chain 3 Iteration: 250 / 450 [55%] (Sampling)
Chain 3 Iteration: 260 / 450 [57%] (Sampling)
Chain 3 Iteration: 270 / 450 [60%] (Sampling)
Chain 3 Iteration: 280 / 450 [62%] (Sampling)
Chain 3 Iteration: 290 / 450 [64%] (Sampling)
Chain 3 Iteration: 300 / 450 [66%] (Sampling)
Chain 3 Iteration: 310 / 450 [68%] (Sampling)
Chain 3 Iteration: 320 / 450 [71%] (Sampling)
Chain 4 Iteration: 310 / 450 [68%] (Sampling)
Chain 4 Iteration: 320 / 450 [71%] (Sampling)
Chain 4 Iteration: 330 / 450 [73%] (Sampling)
Chain 4 Iteration: 340 / 450 [75%] (Sampling)
Chain 4 Iteration: 350 / 450 [77%] (Sampling)
Chain 4 Iteration: 360 / 450 [80%] (Sampling)
Chain 4 Iteration: 370 / 450 [82%] (Sampling)
Chain 4 Iteration: 380 / 450 [84%] (Sampling)
Chain 1 Iteration: 390 / 450 [86%] (Sampling)
Chain 1 Iteration: 400 / 450 [88%] (Sampling)
Chain 1 Iteration: 410 / 450 [91%] (Sampling)
Chain 1 Iteration: 420 / 450 [93%] (Sampling)
Chain 1 Iteration: 430 / 450 [95%] (Sampling)
Chain 1 Iteration: 440 / 450 [97%] (Sampling)
Chain 1 Iteration: 450 / 450 [100%] (Sampling)
Chain 2 Iteration: 250 / 450 [55%] (Sampling)
Chain 2 Iteration: 260 / 450 [57%] (Sampling)
Chain 2 Iteration: 270 / 450 [60%] (Sampling)
Chain 2 Iteration: 280 / 450 [62%] (Sampling)
Chain 2 Iteration: 290 / 450 [64%] (Sampling)
Chain 3 Iteration: 330 / 450 [73%] (Sampling)
Chain 3 Iteration: 340 / 450 [75%] (Sampling)
Chain 3 Iteration: 350 / 450 [77%] (Sampling)
Chain 3 Iteration: 360 / 450 [80%] (Sampling)
Chain 3 Iteration: 370 / 450 [82%] (Sampling)

Chain 3 Iteration: 380 / 450 [84%] (Sampling)
Chain 3 Iteration: 390 / 450 [86%] (Sampling)
Chain 3 Iteration: 400 / 450 [88%] (Sampling)
Chain 4 Iteration: 390 / 450 [86%] (Sampling)
Chain 4 Iteration: 400 / 450 [88%] (Sampling)
Chain 4 Iteration: 410 / 450 [91%] (Sampling)
Chain 4 Iteration: 420 / 450 [93%] (Sampling)
Chain 4 Iteration: 430 / 450 [95%] (Sampling)
Chain 4 Iteration: 440 / 450 [97%] (Sampling)
Chain 4 Iteration: 450 / 450 [100%] (Sampling)
Chain 1 finished in 0.9 seconds.
Chain 4 finished in 0.9 seconds.
Chain 2 Iteration: 300 / 450 [66%] (Sampling)
Chain 2 Iteration: 310 / 450 [68%] (Sampling)
Chain 2 Iteration: 320 / 450 [71%] (Sampling)
Chain 2 Iteration: 330 / 450 [73%] (Sampling)
Chain 3 Iteration: 410 / 450 [91%] (Sampling)
Chain 3 Iteration: 420 / 450 [93%] (Sampling)
Chain 3 Iteration: 430 / 450 [95%] (Sampling)
Chain 3 Iteration: 440 / 450 [97%] (Sampling)
Chain 3 Iteration: 450 / 450 [100%] (Sampling)
Chain 3 finished in 1.0 seconds.
Chain 2 Iteration: 340 / 450 [75%] (Sampling)
Chain 2 Iteration: 350 / 450 [77%] (Sampling)
Chain 2 Iteration: 360 / 450 [80%] (Sampling)
Chain 2 Iteration: 370 / 450 [82%] (Sampling)
Chain 2 Iteration: 380 / 450 [84%] (Sampling)
Chain 2 Iteration: 390 / 450 [86%] (Sampling)
Chain 2 Iteration: 400 / 450 [88%] (Sampling)
Chain 2 Iteration: 410 / 450 [91%] (Sampling)
Chain 2 Iteration: 420 / 450 [93%] (Sampling)
Chain 2 Iteration: 430 / 450 [95%] (Sampling)
Chain 2 Iteration: 440 / 450 [97%] (Sampling)
Chain 2 Iteration: 450 / 450 [100%] (Sampling)
Chain 2 finished in 1.3 seconds.

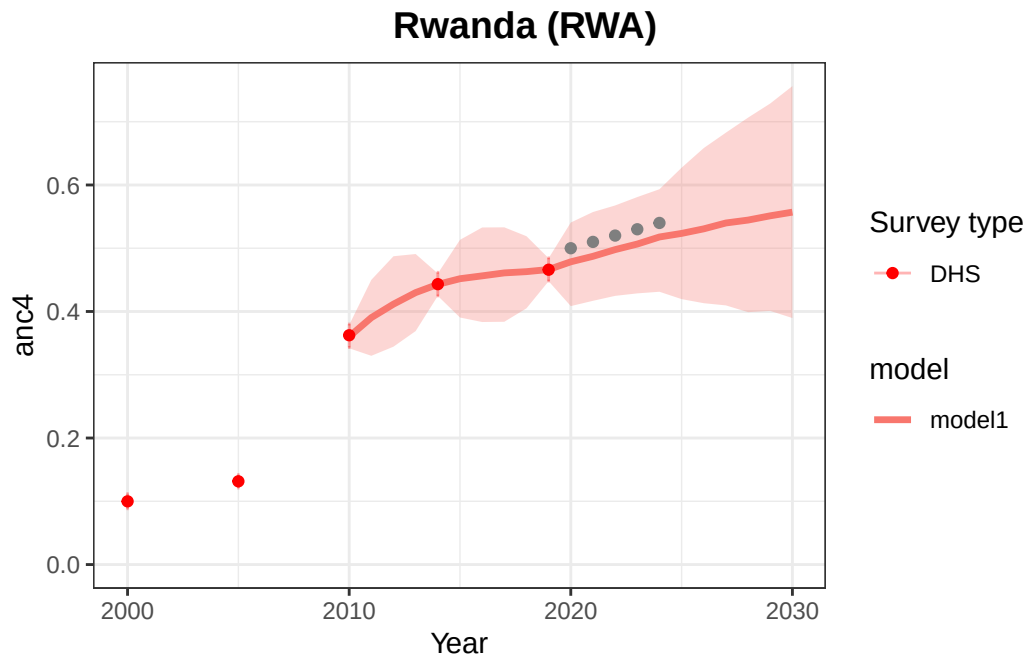
All 4 chains finished successfully.
Mean chain execution time: 1.0 seconds.
Total execution time: 1.4 seconds.

Warning: 1 of 1200 (0.0%) transitions ended with a divergence.
See <https://mc-stan.org/misc/warnings> for details.

Extracting posteriors...

Overview plot

```
countryplot <- plot_estimates_local_all(  
  results = fit,  
  save_plots = FALSE)[[1]]  
countryplot
```



Estimates

```
fit$posteriors$temporal %>%  
  select(iso, year, `50%`, `2.5%`, `97.5%`)
```

```
# A tibble: 21 x 5  
  iso   year `50%` `2.5%` `97.5%`  
  <chr> <int> <dbl> <dbl> <dbl>  
1 RWA   2010 0.360 0.342 0.379  
2 RWA   2011 0.390 0.330 0.450  
3 RWA   2012 0.412 0.344 0.487  
4 RWA   2013 0.430 0.369 0.491  
5 RWA   2014 0.443 0.425 0.460  
6 RWA   2015 0.452 0.390 0.513
```

```
7 RWA      2016 0.456  0.383  0.533
8 RWA      2017 0.461  0.384  0.533
9 RWA      2018 0.463  0.405  0.519
10 RWA     2019 0.466  0.448  0.484
# i 11 more rows
```